

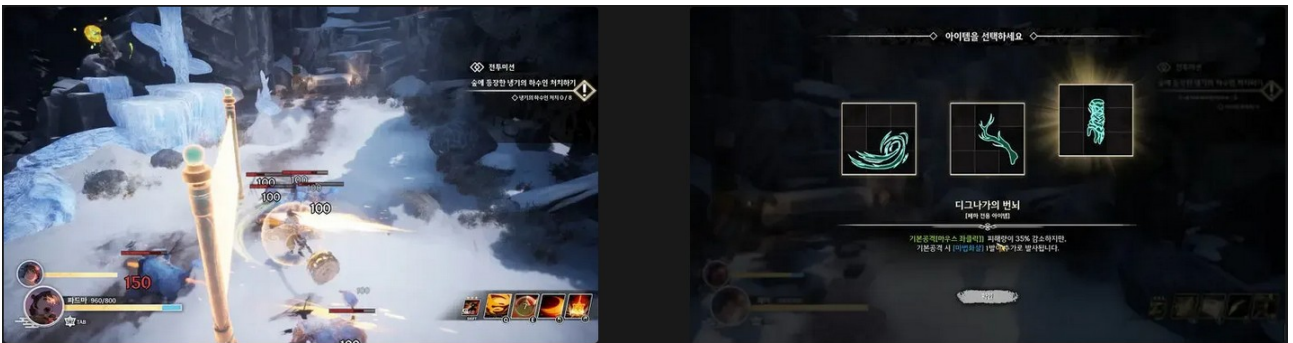
Technical Portfolio

안유찬 | Game Programmer

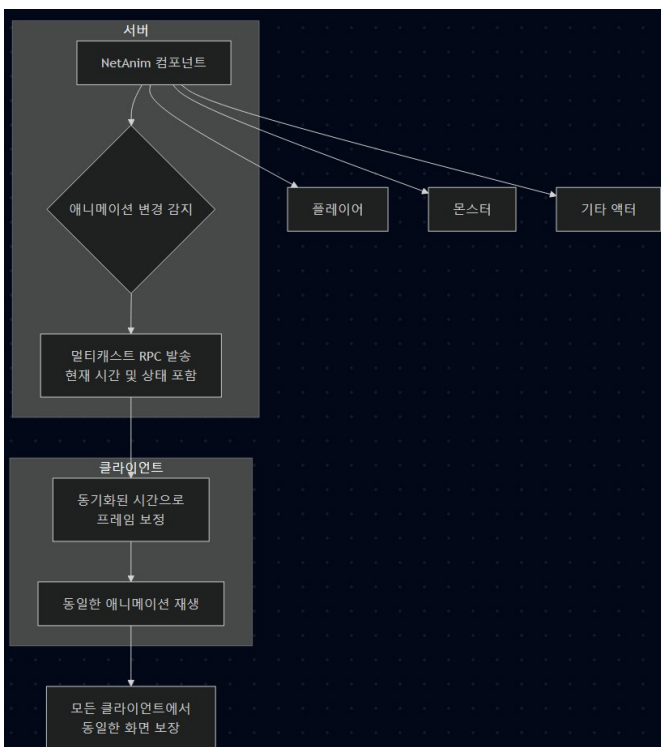
Game Engine System(Unreal, Unity)

Nirvana(Unreal 엔진)

니르바나는 로그라이크 장르의 특성을 티베트 불교의 윤회라는 교리에 비유한 ‘멀티 액션 로그라이크’ 게임입니다. 아이템은 획득한 플레이어 혼자 적용되는 전용 아이템, 2명의 플레이어 모두에게 효과가 적용되는 공용 아이템 총 2가지의 종류가 있습니다. 획득한 아이템을 두 명이 함께 사용하는 하나의 인벤토리에 저장하여 아이템의 능력을 활성화합니다. 아이템들은 각각 고유의 다양한 모양을 가지고 있으며 해당 모양을 인벤토리에 알맞게 배치하여, 한정되어 있는 공간을 전략적으로 협동해서 사용하게 됩니다. 인벤토리의 공간은 스테이지를 클리어할 때마다 확장시킬 수 있습니다.



플레이어 애니메이션 동기화



언리얼 에서 멀티플레이 플레이어의 애니메이션 상태 동기화가 필요합니다. 언리얼 엔진은 액터의 무브먼트에 대해 기본적으로 동기화를 제공합니다. 따라서, 이동(물리)와 관련된 애니메이션에는 동기화가 필요하지 않지만, 액션(타격, 스킬 사용등) 에는 동기화가 필수적입니다.

동기화 되어야 하는 핵심 포인트는 다음과 같습니다.

- 섹션이동
- 재생중 멈춤
- 멈추다가 다시 재생
- 재생중인 몽타주가 바뀜

```
void UNetAnim::NMC_JumpSectionMontage_Implementation(...) {  
    ...  
    AnimInst->Montage_JumpToSection(Name, Montage);  
    AnimInst->Montage_SetPosition(Montage, PrevSectionLength);  
}
```

- 애니메이션 몽타주 섹션 동기화
- 멀티캐스트 RPC 방식으로 전체 동기화 수행

서버와 클라이언트 간의 핑 차이를 고려하여 최초 시간을 동기화하고, 멀티캐스트 RPC를 통해 모든 클라이언트가 동일한 몽타주 섹션을 동일한 시점에 재생하도록 상태를 일치시켰습니다. Montage_SetPosition으로 재생 시작 위치를 맞춰 지연으로 인한 오차를 최소화했습니다.

다만, 지속적인 RTT 변화나 패킷 손실에 대응하는 보간 로직까지는 구현하지 못해, 이는 추후 개선 과제로 남았습니다.

코드: <https://github.com/yuchanahn/-/blob/main/NirvanaCode/NetAnim.cpp>

게임 UI 화면 개발 경험

- 인벤토리 동기화(아이템 선택, 아이템 적용, 우선순위 등...)

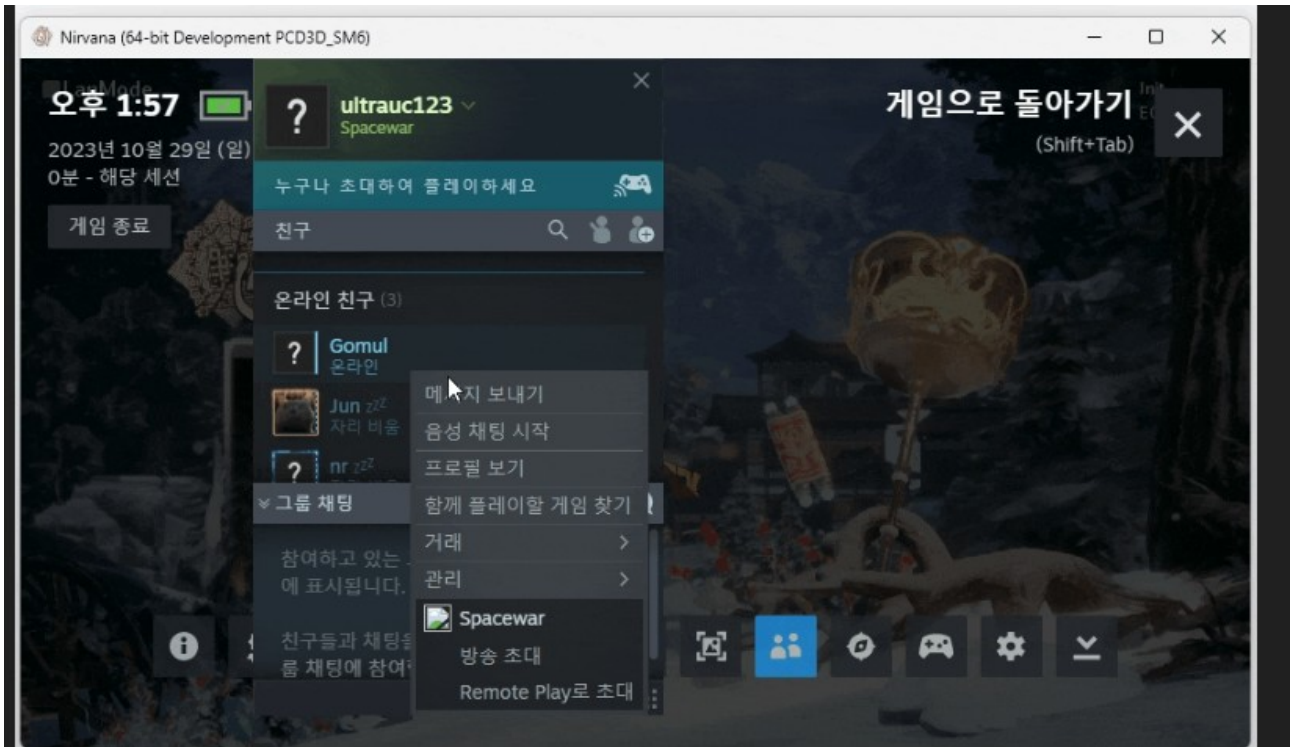
니르바나 게임은 디아블로의 인벤토리처럼 Jigsaw 방식의 인벤토리 시스템을 이용합니다.



두 플레이어가 동시에 아이템을 배치하기 때문에 UI 요소를 실시간으로 동기화 해야 했습니다. 각 플레이어는 그리드의 모든 아이템을 옮길 수 있으며, 아이템을 먼저 집은 플레이어가 옮길 수 권한을 획득하는 등의 로직을 구현했습니다.

코드: <https://github.com/yuchanahn/-/blob/370930eec7c2a739550be131e49cafb99764ef76/NirvanaCode/Core/Client/Client.cpp#L303>

- 로비 화면

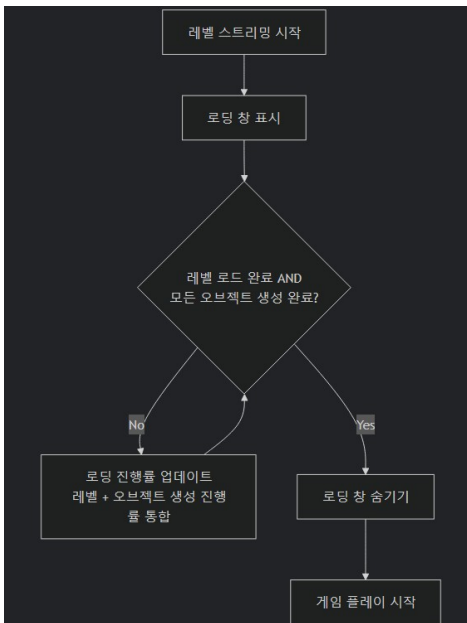


니르바나 게임은 언리얼엔진 리슨서버로 동작하며, 호스트 클라이언트와 보통 클라이언트의 개념이 있습니다.

리슨 서버 구조에 맞게 방 만들기, 방 접속, 친구 초대 등과 함께 로비에서 채팅하는 기능, 캐릭터 바꾸기, 스킬 DB에서 정보 불러와서 보여주는 기능등을 구현하였습니다.

언리얼 엔진의 온라인서브시스템 기능을 이용하여, 스팀 및 EOS에서 플레이 가능하도록 설계 되었으며, 인게임 UI 오버레이, 친구 초대하기 기능을 구현했습니다.

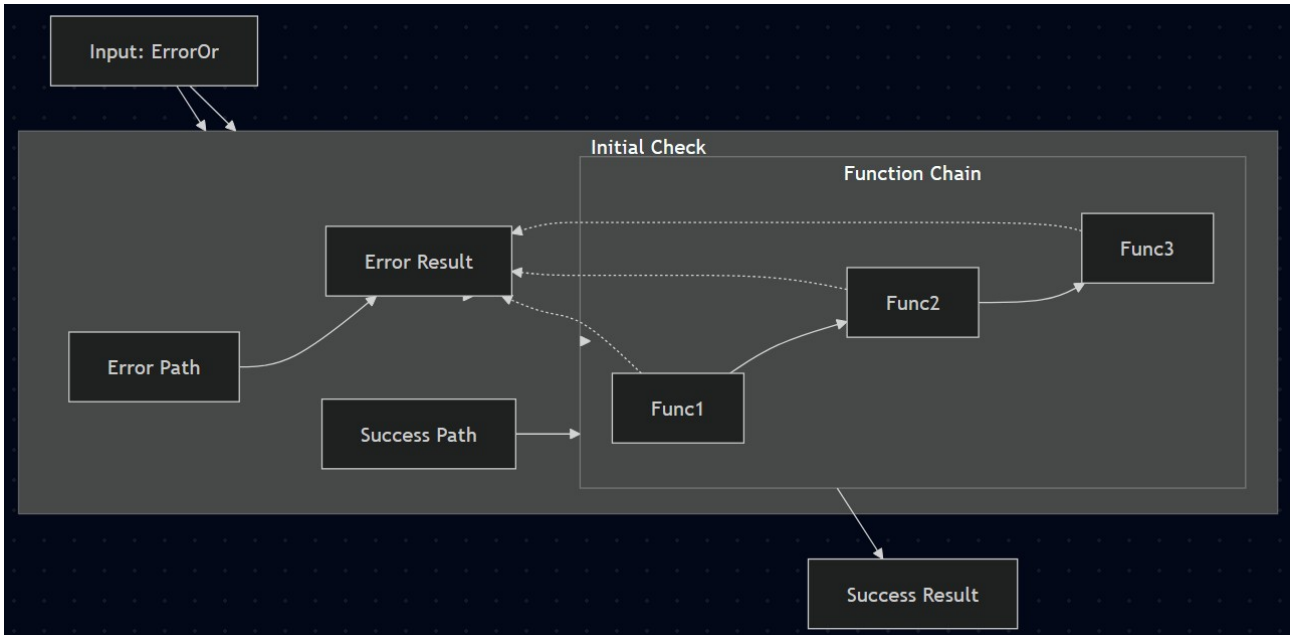
• 로딩 화면



니르바나 게임은 12개의 레벨이 있는 게임으로 빠른 로딩을 위해 언리얼의 레벨 스트리밍 기능을 사용하였습니다. 스트리밍중 발생하는 맵 로드 과정을 가리기 위한, 로딩화면은 레벨 로드가 완료 되었는지 판단하고, 몹 생성기와 서버가 맵에 생성되어야 하는 요소들이 전부 준비 될 때까지, 모든 플레이어가 준비를 마칠때까지 대기하며, 로딩 %를 유저가 확인할 수 있도록 만들었습니다.

언리얼 엔진 안정성 강화를 위한 C++ 에러 핸들링 구조

Rust의 `Result<T, E>`와 함수형 프로그래밍의 체이닝(Chaining) 스타일에 영감을 받아, 중첩된 if 문 없이도 예상된 실패를 깔끔하게 처리할 수 있는 `ErrorOr<T>` 타입을 설계했습니다.



구현 방식: 파이프 연산자 | 오버로딩으로 함수 체이닝

이해를 돕기 위한 코드

```
auto Result = GetSomething() // -> ErrorOr<FMyObject*>
    | Validate                // FMyObject*를 받아 ErrorOr<bool>가 아닌 ErrorOr<FMyObject*>를
그대로 반환
    | Transform              // -> ErrorOr<FTransformedObject*>
    | WhenErr | Log::Error; // 에러 발생 시점에 로그 출력
```

성과: Null 체크를 누락하여 발생하던 에디터 크래시를 컴파일 타임에 방지하고, 여러 단계의 유효성 검사가 필요한 로직을 중첩된 if문 없이 구현하여 코드 가독성과 유지보수성을 향상

코드: <https://github.com/yuchanahn/-/blob/main/NirvanaCode/Core/Utils/Functional.hpp>

코드에서 사용:

```
World.LoadingW | Curry(GetWidgetFromName)(L"Img_BG") | Cast<UImage> | [&](UImage* Texture) {
    Texture->SetBrushFromTexture(Res::T_LoadingPic[0]);
};

World.LoadingBar = World.LoadingW | Curry(GetWidgetFromName)("LoadBar") | Cast<USlider>;
```

코루틴

- C++20 코루틴을 도입, `co_await` 기반의 비동기 타이머를 구현하여 복잡한 Tick 및 델리게이트 기반 지연 로직을 간소화.

개발 편의성

- 게임의 핵심 기능을 신속하게 테스트하기 위한 치트 시스템을 구현했습니다.

- 효율적인 게임 시스템 테스트를 위해 레벨(Map) 환경과 DA를 이용하여 게임의 상태를 관리(플레이어 스탯, 싱글/멀티, UI테스트 등등)

언리얼 엔진으로 협업 시 빌드/패키징 경험

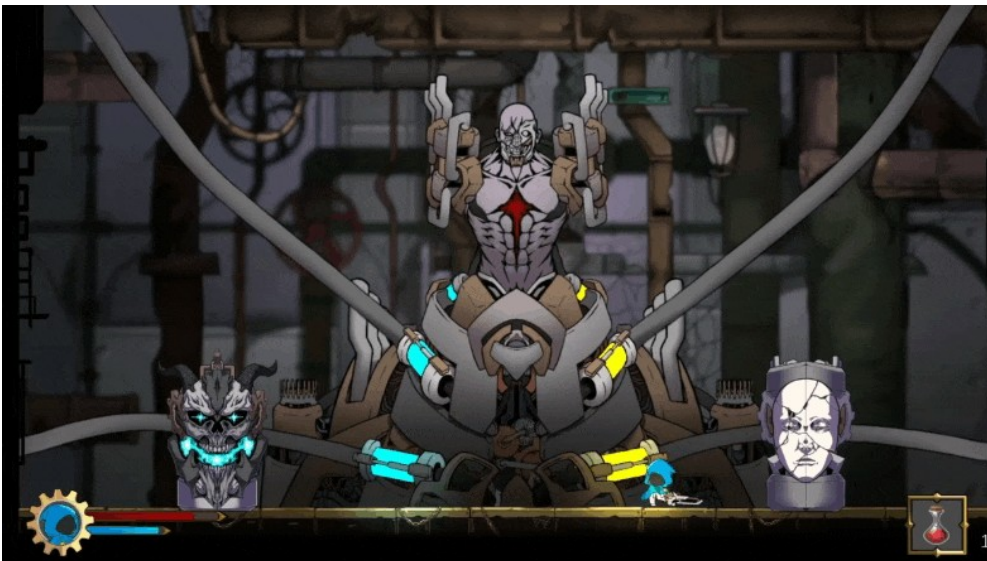
- Nirvana 프로젝트를 진행하며, 5.1 -> 5.3 버전 업그레이드 경험
- 언리얼 엔진 다양한 패키징 실패 사례를 해결하며, 빌드 프로세스에 대한 깊은 이해를 얻었습니다.
- 프로그래밍이 아닌 다른 파트 팀원들을 위한, 바이너리 파일 동기화(git, 간단한 배치 스크립트) 솔루션 설계.

프로젝트 회고

- 프로젝트 초기, 언리얼 엔진의 표준 방식 대신 저만의 C++ 중심 아키텍처를 시도했습니다. 이를 통해 C++ 자체에 대한 이해도는 높일 수 있었지만, 언리얼의 게임플레이 프레임워크와 유기적으로 동작하는 데에는 비효율이 발생했습니다. 이 경험을 통해 엔진이 제공하는 표준 아키텍처를 깊이 이해하고 활용하는 것이 장기적인 유지보수와 팀 생산성에 얼마나 중요한지 값진 교훈을 얻을 수 있었습니다.

Vapor(Unity: 2D 액션 게임)

C#과 Unity URP를 기반으로 2D 횡스크롤 액션 어드벤처 게임을 개발하며 엔진의 핵심 기능을 깊이 있게 활용했습니다. 시스템 아키텍처 설계, 성능 최적화, 개발 파이프라인 구축에 중점을 두었습니다.



시스템 아키텍처 설계

게임의 각 기능을 독립적인 매니저(Manager)로 분리하여 관리하는 모듈식 아키텍처를 채택했습니다. 이를 통해 시스템 간 의존성을 낮추고 코드의 유지보수성과 확장성을 확보했습니다.

- **GameManager**, **UIManager**, **SoundManager** 등 싱글톤 패턴 기반의 핵심 매니저들을 구현하여 게임의 상태, UI, 사운드, 씬 전환 등 주요 로직을 중앙에서 관리하도록 설계했습니다.
- 총알, 이펙트 등 빈번하게 생성/삭제되는 오브젝트를 위해 **오브젝트 풀링(objPool)** 시스템을 구현하여, 런타임 중 메모리 할당 및 가비지 컬렉션으로 인한 성능 저하를 최소화했습니다.

확장성을 고려한 전투 시스템 구현

복잡한 전투 로직을 체계적으로 관리하기 위해 디자인 패턴을 적극적으로 활용했습니다.

- **상태 패턴(State Pattern)**을 적용하여 플레이어와 AI의 행동(공격, 피격, 패링, 차지 등)을 각 상태 클래스로 분리했습니다. 이를 통해 복잡한 공격 메커니즘을 명확하게 구현하고 신규 상태 추가가 용이한 구조를 만들었습니다.
- **HitBox와 IHitable 인터페이스** 기반의 피격 시스템을 설계하여, 공격 판정과 피격 처리를 분리했습니다. 이 구조 덕분에 다양한 종류의 무기, 스킬, 함정 등 공격 주체와 캐릭터, 파괴 가능 오브젝트 등 피격 객체를 유연하게 추가할 수 있었습니다.
- **코드 (플레이어 상태 및 컨트롤)**: <https://github.com/yuchanahn/-/blob/main/UnityScripts/Player/PC.cs>
- **코드 (UI 및 연출)**: <https://github.com/yuchanahn/-/tree/main/UnityScripts/UI>

데이터 관리 파이프라인 구축

기획자와의 원활한 협업 및 개발 효율성 증대를 위해 데이터 관리 시스템을 구축했습니다.

- **Google Sheets 연동**: UnityWebRequest를 사용하여 Google Sheets의 CSV 데이터를 런타임에 불러와 파싱하는 시스템을 구현했습니다. 이를 통해 기획자가 코드를 직접 수정하지 않고도 몬스터 스탯, 아이템 정보 등 게임 밸런스를 실시간으로 조절할 수 있는 데이터 파이프라인을 마련했습니다.
- **파일 기반 저장/불러오기**: 플레이어의 진행 상황, 설정 값 등을 JSON으로 직렬화하여 로컬 파일에 저장하고 불러오는 시스템을 구현했습니다. 이를 통해 데이터의 안정적인 유지를 보장했습니다.

Low-Level System & Backend Engineering

H/W·OS 네트워크 프로그래밍 저수준 경험입니다.

네트워킹 라이브러리 (C++)

- UDP 기반으로 신뢰성 있는 전송을 테스트하기 위해 ACK를 주고받는 기초적인 RUDP 프로토콜을 구현했습니다.

코드(RUDP): <https://github.com/yuchanahn/ycnet/blob/main/src/ycnet.hpp>

넷코드

Rust언어와 Godot엔진을 활용하여, 홀펀칭 구현과 롤백 넷코드 데모 예제(격투게임)

- 딜레이 기반 넷코드 (Lockstep 구조)
 - ▶ 모든 인풋은 몇 틱 뒤에 실행될 시간 정보(tick) 와 함께 전송
 - ▶ 로컬에서는 미리 입력을 저장하고, 예정된 틱에 실행
 - ▶ 인풋이 들어오지 않으면 게임 자체를 일부러 지연 시켜 동기화 보장
- 롤백 넷코드
 - ▶ 가장 최근의 입력으로 예측 실행
 - ▶ 예측이 틀릴 경우, 과거 상태로 롤백 후 재실행
 - ▶ 예측된 인풋은 캐시에 저장되고, 실제 입력이 도착하면 비교 후 처리



코드(롤백): https://github.com/yuchanahn/-/blob/main/GodotP2PGame/src/network_controller.rs

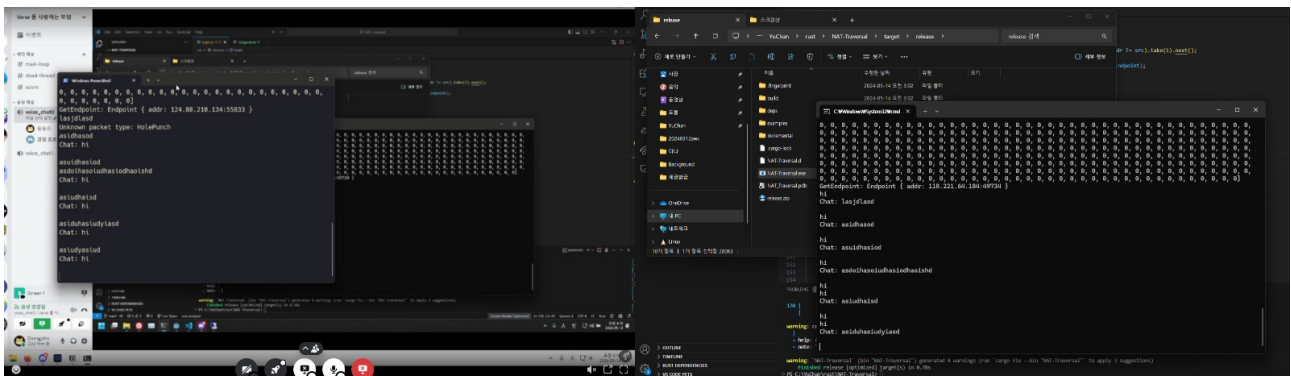
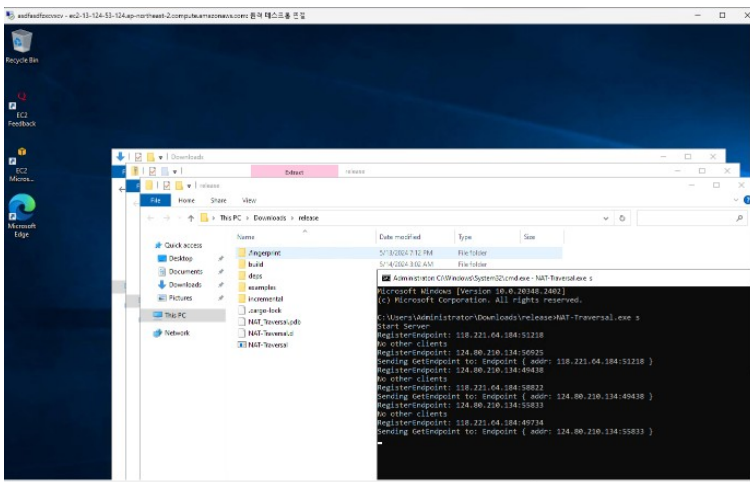
테스트1: Clumsy 툴로 450ms 지연에서 격투 게임 플레이

테스트2: 2대의 컴퓨터로 롤백 테스트

원활한 플레이'보다는 '입력 지연 없는 반응성'을 목표로 롤백 네트코드를 구현했습니다. 450ms의 극심한 지연 환경에서도 플레이어의 입력이 즉각적으로 화면에 반영되는 것을 확인하며, 롤백 네트코드의 핵심 장점을 직접 구현하고 검증할 수 있었습니다.

출판칭

- 중계 서버 및 출판칭 구현(AWS 인스턴스에 중계 서버 실행)



코드: <https://github.com/yuchanahn/-/blob/main/GodotP2PGame/HolePunching.rs>

실시간 자동매매 시스템 (D, Nim, C++)

- Bybit API 연동, 백테스팅을 위해 파이썬 시각화 라이브러리 활용.
- 네트워크 오류 대응과 프로세스 자동 복구 로직 → 24시간 무중단 목표로 운영

코드(Cpp): https://github.com/yuchanahn/trade_bot_bybit_cpp/blob/master/trade_bot_bybit/main.cpp

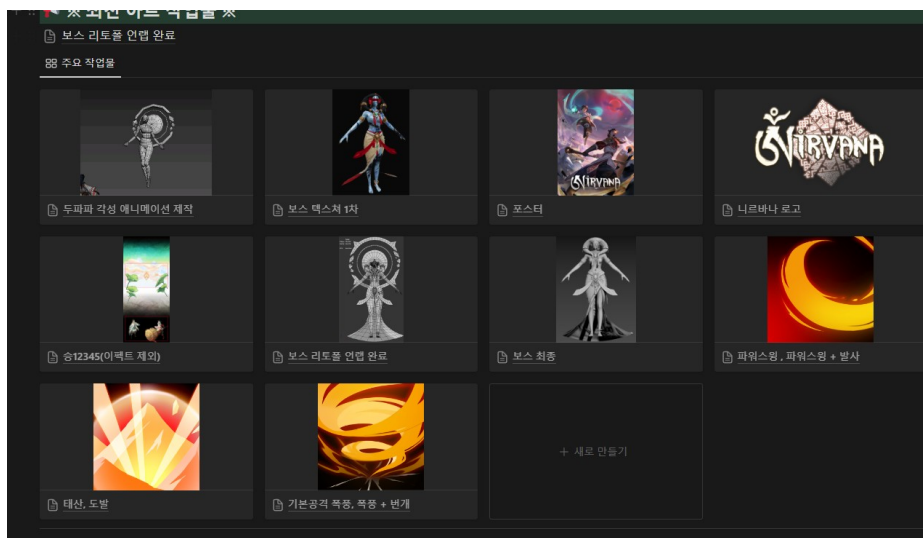
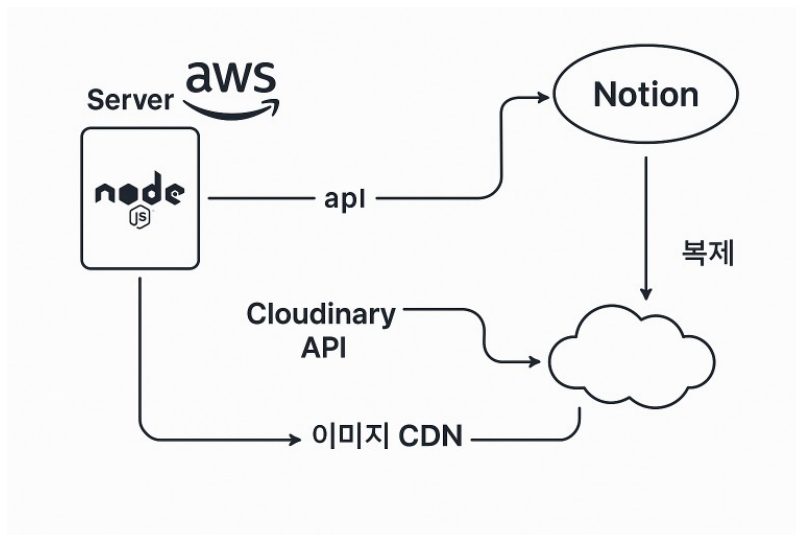
코드(D): <https://github.com/yuchanahn/-/blob/main/D/source/trade.d>

Development Productivity & Automation

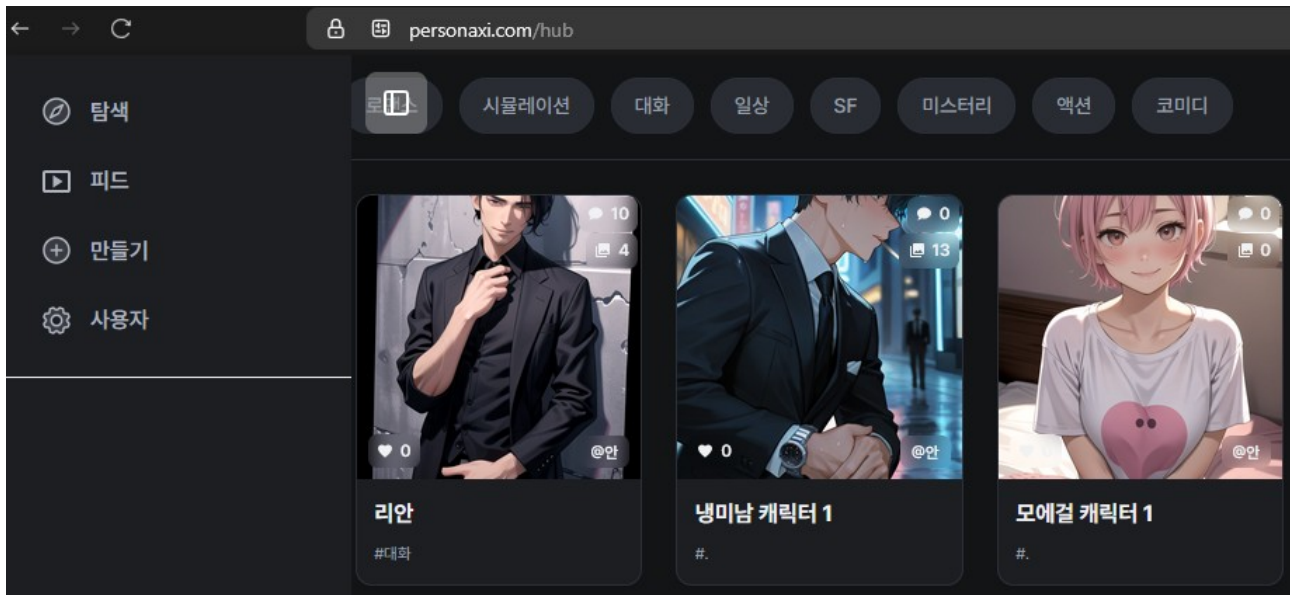
Notion 활용 협업 자동화 툴 (Node.js)

Nirvana 게임의 팀 작업을 초기, 효율적인 작업상황 공유를 위하여, 각 팀원들의 자체 노션 DB에서 최신 정보를 가져와 팀 워크스페이스 대시보드에 정보를 표시하는 작업을 진행.

- 노션 API로 분산된 작업 현황을 통합, 실시간 대시보드화
- Cloudinary API로 이미지 업로드 제약 해결 + 비용 최적화



Project: personaxi.com (Full-Stack Web)



Personaxi는 단순한 챗봇 서비스를 넘어, 사용자와 실시간으로 상호작용하며 감정적 유대를 형성하는 **감정 기반 디지털 생명체**를 구현하는 프로젝트이며, 핵심 목표는 크리에이터가 자신의 AI 페르소나를 창조하고, 방송 및 경매 시스템을 통해 수익을 창출할 수 있는 독자적인 생태계를 구축입니다.

- **핵심 비전:** LLM, 감정 분석, VRM 립싱크 기술을 융합하여 살아있는 듯한 AI 경험 제공
- **주요 특징:** 크리에이터 경제 시스템 (방송, 경매), 실시간 음성 대화 (STT/TTS), 27가지 감정 스펙트럼 분석 기반의 동적 상호작용
- **기대 효과:** 사용자에게 높은 몰입감을 제공하고, 크리에이터에게는 새로운 창작 및 수익화의 장을 마련

기술 스택 및 아키텍처 (Tech Stack & Architecture)

프로젝트는 확장성과 안정성, 빠른 개발 속도를 고려하여 최신 기술 스택을 적극적으로 도입하였습니다.

Backend:

- Go: 확장성 및 동시성 처리에 강점을 가진 서버 개발 언어
- PostgreSQL (AWS RDS): 안정적인 데이터 저장을 위한 관계형 데이터베이스
- Redis: 캐싱, 실시간 데이터 처리 등 빠른 속도가 요구되는 작업을 위한 인메모리 데이터베이스
- Custom Middleware: Rate Limiting(요청 제한) 등 시스템 안정성 및 보안 강화를 위한 자체 미들웨어 구현

Frontend

- Svelte & TypeScript: 간결하고 반응적인 UI를 구축하기 위한 최신 프레임워크와 타입스크립트 조합
- Three.js & VRM: 3D 모델링 및 가상현실(VR) 콘텐츠를 웹에서 구현
- Tailwind CSS: 유틸리티-우선(Utility-First) 접근법을 통한 신속하고 효율적인 UI 스타일링
- PWA (Progressive Web App): 설치형 앱처럼 동작하면서도 웹 기반으로 접근 가능한 하이브리드 앱 구조.
 - ▶ 오프라인 접근성, 푸시 알림, 홈 화면 설치 기능을 통해 사용자 경험(UX) 강화
 - ▶ React Native나 Flutter에 비해 유지보수가 간단하고 빠른 배포 가능

AI & Machine Learning

Generative AI APIs:

- Google Gemini & OpenAI (via OpenRouter): 최신 언어 모델을 활용한 지능형 서비스 구현
- ElevenLabs API: 고품질 음성 합성을 통한 TTS(Text-to-Speech) 기능

AI/ML Platforms & Models:

- Hugging Face (Spaces, API): 최신 AI 모델 배포, 서빙 및 API 연동
- Ollama: 로컬 환경에서의 대규모 언어 모델(LLM) 구동 및 테스트
- KoELECTRA (Docker): 한국어 특화 감정 분석 모델 활용 (GCP에서 Hugging Face로 이전)

Vector Database:

- pgvector: PostgreSQL 확장 기능으로, 캐릭터 기억구현을 위해 사용

DevOps & Infrastructure

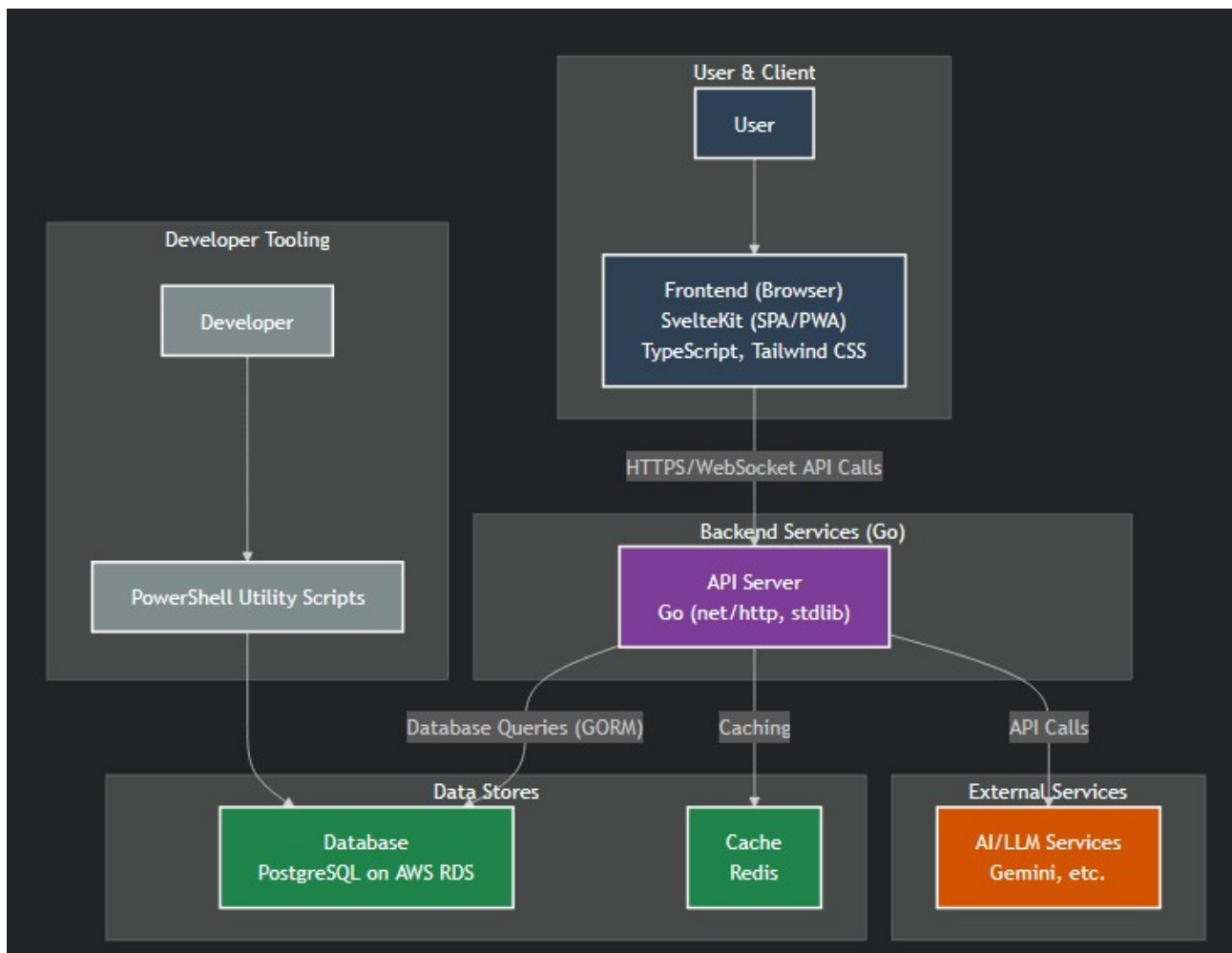
- Docker: 애플리케이션의 일관된 배포 및 실행 환경 구축
- GitHub Actions: 코드 통합(CI) 및 배포(CD) 자동화를 통한 개발 생산성 향상
- GCP (Google Cloud Platform): 과거 감정 분석 모델 호스팅 경험
- Load Balancer: 트래픽 분산을 통한 시스템 안정성 및 가용성 확보

Authentication & Real-time Communication

- JWT (JSON Web Token): 안전하고 효율적인 사용자 인증 및 인가 시스템
- OAuth 2.0 (Google): 소셜 로그인을 통한 간편하고 안전한 사용자 인증
- WebSocket: 실시간 양방향 통신을 기반으로 한 STT/TTS 기능 구현
- Email Verification: 이메일을 통한 사용자 계정 인증 시스템

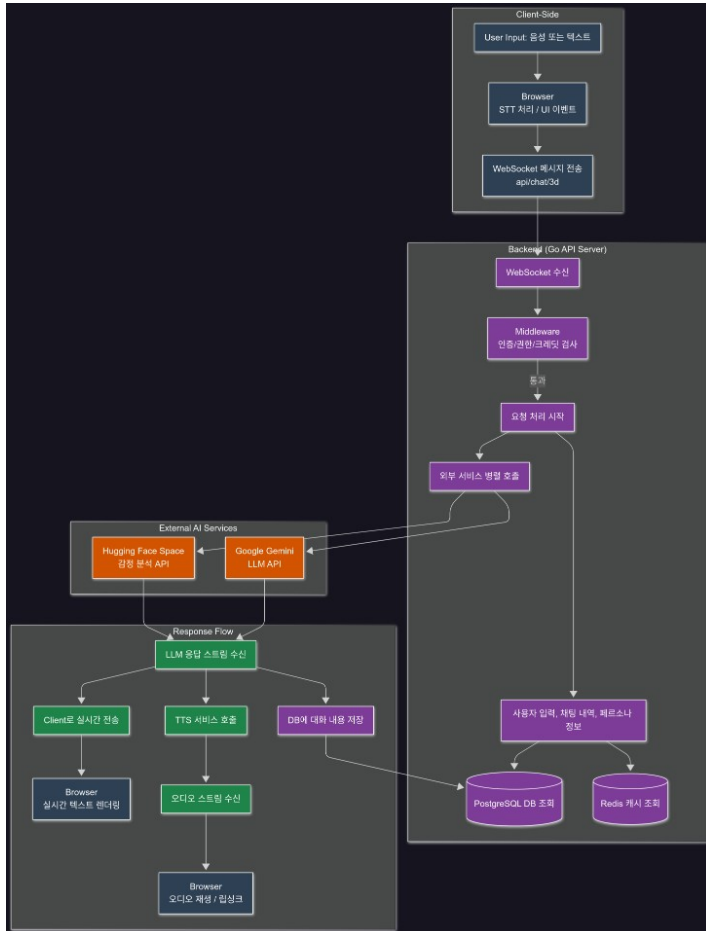
상세 개발 정보

Backend (Go & Python)



• 고성능 API 서버 구축 (Go):

- ▶ RESTful API 설계: 사용자 인증(JWT), 페르소나 CRUD, 경매/방송 로직 등
- ▶ 동시성 처리: 고루틴(Goroutine)과 채널(Channel)을 활용한 대규모 트래픽 처리 및 외부 API 호출 비동기화
- ▶ 데이터베이스 연동: database/sql 및 pq를 이용한 PostgreSQL(RDS) 연동
- ▶ API 호출 구조도



• 안정성 및 운영:

- ▶ 네트워크 오류 대응: 외부 API 호출 실패 시 재시도(Retry) 로직



- ▶ 프로세스 자동 복구: 예상치 못한 오류로 서버 다운 시, 시스템 서비스(systemd 등)를 통한 자동 재시작 로직 구현

- 도커

- ▶ 외부모델 업로드 및 엔드포인트 활성화.



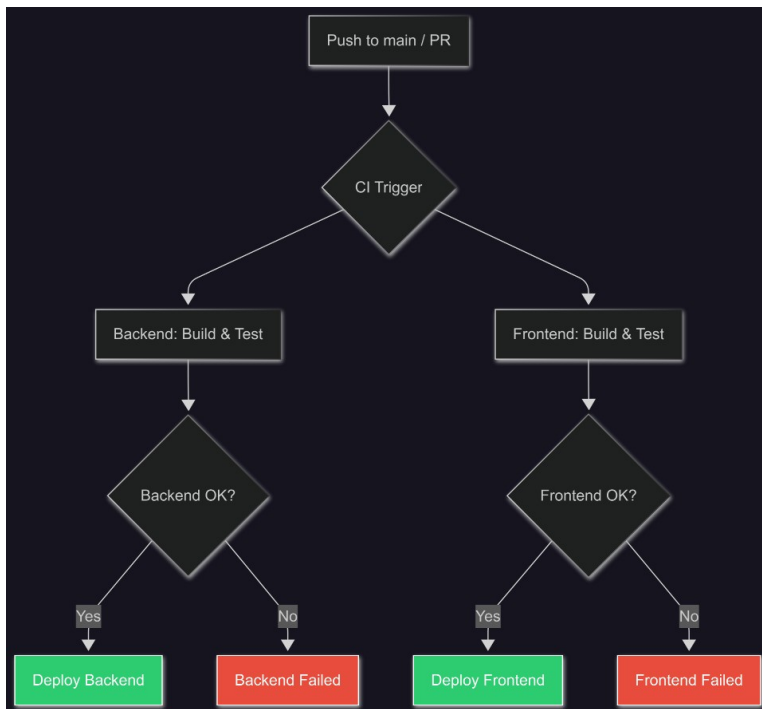
Frontend (SvelteKit)

- 인터랙티브 UI/UX 구현:

- ▶ WebSocket 활용: WebSocket 기반 STT/TTS 데이터 양방향 스트리밍
- ▶ 립싱크: three.js VRM 뷰어를 통한 감정 및 립싱크 실시간 렌더링
- ▶ 반응형 디자인: 모바일/데스크톱 크로스 플랫폼 UI 제공
- ▶ 효율적 상태 관리: Svelte 스토어를 활용한 전역 상태(사용자, 페르소나, 대화) 관리

DevOps (CI/CD)

• CI (GitHub Actions):



pages build and deployment #156

Summary

Jobs

- build
- report-build-status
- deploy

Run details

Usage

Triggered via GitHub Pages 9 hours ago

github-pages[bot] -> becbca7 gh-pages

Status: Success

Total duration: 27s

Artifacts: 1

pages-build-deployment

on: dynamic

build (10s) → report-build-status (3s) → deploy (9s)

deploy: http://personaxi.com/

- Go 백엔드: 코드 푸시 시 자동 빌드.

- SvelteKit 프론트엔드: 의존성 설치 및 프로덕션 빌드 검증

• CD (Custom Shell Script on EC2):

- ▶ 재시작 스크립트(deploy.sh) 운영, Blue-Green 배포는 비용 문제로 미적용
- ▶ 배포 파이프라인: git pull → 기존 프로세스 안전 종료 → 신규 빌드 → 서비스 재시작

• 성과

- ▶ 기능별 역할을 명확히 분리하기 위해, 주 비즈니스 로직을 처리하는 Go 서버와 한국어 감정 분석을 전담하는 Python 모델 서버로 구성된 아키텍처를 구축했습니다.